Quantum Natural Language Processing for Chess Annotations

A Hybrid Quantum–Classical Approach to Tactical Motif Classification

Sandeep Cherukuri, Shreyas Deosthali, Soham Date

October 8, 2026

Abstract

Chess annotations describe positions in natural language, yet most computational chess work ignores this text. This project builds an end-to-end system that classifies annotated chess moves into tactical motifs: pin, fork, sacrifice, blunder, or none. Each annotation is parsed into a compositional DisCoCat diagram and compiled into a parameterized quantum circuit, while the board position is encoded as a 768-dimensional vector. A hybrid network built with PyTorch, PennyLane and `lambeq` fuses the two representations to predict the motif. We describe the data pipeline, the linguistic and quantum modelling stages, and the model architecture, and outline directions for extending the work toward annotation style transfer.

Contents

1 Introduction **3**
- 1.1 Objectives . . . 3
- 1.2 Problem statement . . . 3

2 Background **3**
- 2.1 Chess data formats . . . 3
- 2.2 Compositional distributional semantics . . . 3
- 2.3 Quantum natural language processing . . . 4

3 System Architecture **4**

4 Data Pipeline **5**
- 4.1 Corpus loading . . . 5
- 4.2 Cleaning . . . 5
- 4.3 Exploratory analysis . . . 5
- 4.4 Motif labeling . . . 5
- 4.5 Board encoding . . . 5
- 4.6 Dataset construction . . . 5

5 Linguistic Processing **6**

6 Quantum Circuit Construction **6**
- 6.1 Diagram simplification . . . 6
- 6.2 Ansatz . . . 6
- 6.3 Circuit output . . . 6

7 Hybrid Quantum–Classical Model **6**
7.1 Architecture 6
7.2 Training 6
7.3 Evaluation 7

8 Future Work **7**

9 Conclusion **7**

1 Introduction

Players and engines annotate games with short comments such as "This pins the knight to the king." A collection of annotated games is therefore a collection of pairs: a board position and a sentence about it. This project asks whether a language model that respects sentence grammar, implemented as a quantum circuit, can recover the tactical idea an annotator describes.

1.1 Objectives

- Build a reproducible pipeline from raw annotated PGN games to a model-ready dataset.
- Represent annotation sentences as grammar-aware DisCoCat diagrams and quantum circuits.
- Represent board positions as numerical vectors.
- Combine both inputs in a hybrid quantum–classical network that classifies tactical motifs.

1.2 Problem statement

Given a comment and the board position it refers to, predict one of five classes (Table 1), encoded as $none = 0$, $pin = 1$, $fork = 2$, $sacrifice = 3$, $blunder = 4$.

Motif	Description
Pin	A piece cannot or should not move because it would expose a more valuable piece behind it.
Fork	One piece attacks two or more enemy pieces at once.
Sacrifice	A player deliberately gives up material for a greater gain.
Blunder	A serious mistake.
None	The comment describes none of the above.

Table 1: Tactical motif classes.

2 Background

2.1 Chess data formats

PGN (Portable Game Notation) stores a game as headers and a move list, with comments in curly braces after the move they discuss. Comments may also hold machine commands such as `[%clk 0:05:00]` (clock) and `[%eval 0.30]` (engine evaluation).

FEN (Forsyth–Edwards Notation) encodes a single position as six fields: piece placement, side to move, castling rights, en passant square, halfmove clock and fullmove number. The starting position is

```
rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1
```

2.2 Compositional distributional semantics

Counting words cannot distinguish "rook pins knight" from "knight pins rook". The DisCoCat framework of Coecke, Sadrzadeh and Clark [1] combines distributional word meaning with grammar, so that grammatical structure determines how word meanings combine into a sentence meaning. Words receive pregroup types built from n (noun) and s (sentence); each type x has adjoints x^l and x^r with cancellation rules

$$x \cdot x^r \to 1, \qquad x^l \cdot x \to 1.$$

A transitive verb has type $n^r \cdot s \cdot n^l$, so for "Rook pins knight":

$$n \cdot (n^r \cdot s \cdot n^l) \cdot n \;\to\; s.$$

Each cancellation is drawn as a *cup*, and the derivation is a string diagram, which can be interpreted as a tensor network.

2.3 Quantum natural language processing

In QNLP, a string diagram is converted into a parameterized quantum circuit. Each basic type is assigned a number of qubits, and each word becomes a block of gates whose rotation angles are trainable parameters. Measuring the circuit yields outcome probabilities that serve as the sentence representation. The `lambeq` library [2] provides parsing, diagram rewriting and circuit construction, and its PennyLane integration [3] allows the circuits to be trained with PyTorch.

3 System Architecture

The system has two parts. Part 1 (folder `LInguistic Parsing`) prepares the data and produces diagrams, board vectors and labels. Part 2 (repository root) compiles circuits, defines the hybrid model and trains it.

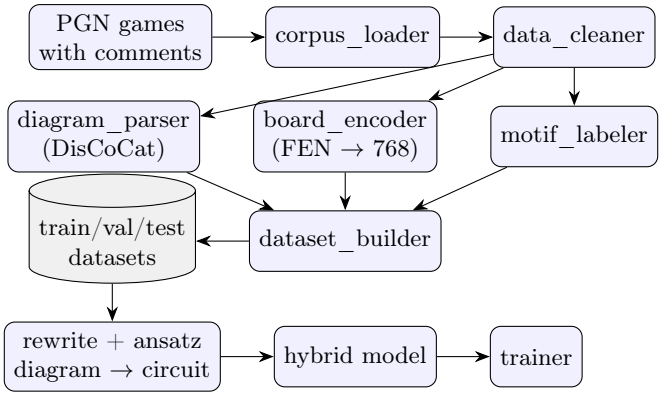


Figure 1: System pipeline from raw annotated games to a trained classifier.

Module	Responsibility
`corpus_loader.py`	Extracts, for each annotated move, the board position, the comment and its source.
`data_cleaner.py`	Removes clock times, engine evaluations and other annotation markup; removes duplicates; counts words.
`EDA_report.py`	Exploratory analysis of the cleaned dataset.
`motif_labeler.py`	Assigns and numerically encodes the motif label.
`diagram_parser.py`	Converts each sentence into a DisCoCat diagram.
`board_encoder.py`	Converts a FEN position into a numerical vector.
`dataset_builder.py`	Assembles `(diagram, board_vector, label)` triples and splits the data.
`pipeline.py`	Runs Part 1 from raw games to the final dataset.
`data_rewriter.py`	Simplifies diagrams before circuit construction.
`ansatz_converter.py`	Converts diagrams into parameterized quantum circuits.
`qnlp_model.py`	Defines the hybrid quantum–classical network.
`trainer.py`	Training and evaluation.

Table 2: Repository modules.

4 Data Pipeline

4.1 Corpus loading

For every move that carries a comment, the loader records the source file, the move, the comment, and the position after the move. Storing the post-move position matches how annotators describe the consequence of a move.

4.2 Cleaning

The cleaner removes any bracketed engine command matching `\[%.*?\]`, collapses repeated whitespace, discards rows whose comment becomes empty, removes duplicate (comment, position) pairs, and adds a word-count feature. For example,

```
[%clk 0:05:00] [%eval 0.30] White opens with the king pawn.
```

becomes "White opens with the king pawn." (six words).

4.3 Exploratory analysis

The EDA module reports the number of rows, the distribution of comment lengths (mean, spread, quartiles, extremes), missing values, duplicate positions, and the number of rows per source file.

4.4 Motif labeling

Labels are assigned by a keyword rule applied to the lowercased comment, checked in the order blunder, pin, fork, sacrifice:

```
MOTIF_KEYWORDS = {
    "blunder":   ["blunder", "??", "loses␣material", "mistake"],
    "pin":       ["pin", "pinned"],
    "fork":      ["fork", "forking"],
    "sacrifice": ["sac", "sacrifice", "sacrificing"],
}
```

Comments with no match are labeled *none*. This provides an initial labeled dataset that can later be refined with manual annotation or position-based verification.

4.5 Board encoding

Each position is encoded as a 768-dimensional binary vector: 64 squares $\times$ 12 piece types (six types in two colors), one-hot per square. The index of a piece is

$$\text{idx} = 12 \cdot \text{square} + \text{plane}, \quad \text{plane} = (\text{type} - 1) + \begin{cases} 0 & \text{White} \\ 6 & \text{Black} \end{cases}$$

with squares numbered $a1 = 0, \ldots, h8 = 63$ and types PAWN= $1, \ldots,$KING= 6. For example, the white rook on a1 maps to index 3, the white king on e1 to 53, and the black king on e8 to 731. The starting position has exactly 32 active entries.

4.6 Dataset construction

The dataset builder combines diagram, board vector and label into triples, shuffles them, and splits them 70%/15%/15% into training, validation and test sets stored for Part 2.

5 Linguistic Processing

`diagram_parser.py` converts each cleaned comment into a DisCoCat string diagram using `lambeq`'s `BobcatParser`, a trained grammar parser that assigns pregroup types to words. The diagram for "Rook pins knight" is built in `lambeq` as

```
rook   = Word("Rook",   N)
pins   = Word("pins",   N.r @ S @ N.l)
knight = Word("knight", N)
sent = rook @ pins @ knight >> Cup(N, N.r) @ Id(S) @ Cup(N.l, N)
```

A type inventory counts the output types appearing across all diagrams, giving the circuit stage the information needed to assign qubits.

6 Quantum Circuit Construction

6.1 Diagram simplification

Cups in a circuit are realized through post-selection, which costs extra qubits and discarded runs. The rewriter applies `lambeq`'s `RemoveCupsRewriter` before circuit construction, bending the noun boxes so that no cups remain. For "Rook pins knight" this reduces the circuit from 5 qubits and 22 gates to 3 qubits and 16 gates, with the output type s preserved.

6.2 Ansatz

The `CircuitCompiler` assigns one qubit to each of the noun and sentence types and supports two ansatz families from `lambeq`:

- **IQP:** alternating layers of Hadamard gates and diagonal unitaries built from controlled-Z rotations.
- **Sim14:** two rings of controlled-X rotations in opposite orientation.

The pipeline uses an IQP ansatz with one layer. Each word is assigned its own set of trainable angles, so the number of quantum parameters grows with vocabulary size.

6.3 Circuit output

Each circuit is simulated through `PennyLaneModel` and returns normalized outcome probabilities. With a one-qubit sentence output, each comment yields a two-element probability vector.

7 Hybrid Quantum–Classical Model

7.1 Architecture

The model `chessQNLPModel` processes the two inputs in parallel branches and fuses them:

- **Sentence branch:** circuit probabilities (2) $\to$ Linear($2 \to 64$) $\to$ LayerNorm $\to$ GELU.
- **Board branch:** board vector (768) $\to$ Linear($768 \to 128$) $\to$ LayerNorm $\to$ GELU.
- **Fusion and classification:** concatenation (192) $\to$ Linear($192 \to 5$), producing one score per class.

Using both inputs lets the classifier combine what the annotator wrote with what is actually on the board.

7.2 Training

Class scores are converted to probabilities with softmax and optimized with cross-entropy loss. Parameters are updated by backpropagation using the Adam optimizer with learning rate 10^{-3}.

Through the `lambeq`–PennyLane integration, gradients flow through the quantum circuits, so circuit angles and network weights are trained jointly. Training runs for 10 epochs by default, using the full training set at each step.

7.3 Evaluation

Because annotated corpora typically contain many more *none* comments than any single motif, accuracy alone is insufficient. The trainer therefore reports a per-class classification report and a confusion matrix.

- **Precision:** of the comments predicted as a motif, the fraction that truly belong to it.
- **Recall:** of the comments truly belonging to a motif, the fraction found.
- **F1:** the harmonic mean of precision and recall.
- **Confusion matrix:** true versus predicted class, showing which motifs are confused.

Training uses the training split, the validation split monitors overfitting, and the held-out test split gives the final score.

8 Future Work

- **Annotation style transfer:** extend the system from classification to rewriting annotations between styles, the long-term goal reflected in the project title.
- **Richer labels:** refine the keyword labels with manual annotation and verify tactical motifs directly against board geometry.
- **Game-level splits:** record a game identifier for each move so that train, validation and test sets can be separated by game.
- **Richer board features:** add side to move, castling rights and en passant information to the board vector.
- **Ablation studies:** compare board-only, sentence-only and combined models to measure the contribution of each input and of the quantum circuit.
- **Larger sentence representation:** assign two or more qubits to the sentence type to widen the circuit output.

9 Conclusion

This project delivers a complete pipeline that turns annotated chess games into a model-ready dataset and a hybrid quantum–classical network for tactical motif classification. It combines grammar-aware quantum circuits for natural language with a one-hot board encoding in a single trainable model, and it provides a foundation for studying how quantum NLP methods perform on a specialized, structured domain.

References

[1] B. Coecke, M. Sadrzadeh, S. Clark. Mathematical Foundations for a Compositional Distributional Model of Meaning, 2010. https://arxiv.org/abs/1003.4394

[2] D. Kartsaklis et al. lambeq: An Efficient High-Level Python Library for Quantum NLP. https://arxiv.org/pdf/2110.04236

[3] PennyLane. Quantum NLP with the lambeq-PennyLane integration, 2023. https://pennylane.ai/blog/2023/04/quantum-nlp-with-the-lambeq-pennylane-integration/

[4] python-chess documentation, PGN parsing. https://python-chess.readthedocs.io/en/latest/pgn.html

[5] Capstone_Project repository. https://github.com/ShreyasD21/Capstone_Project